# Power BI Study Guide: Lessons 17–50

Lesson definitions • Practical DAX examples • Usage notes

## Before you start

Examples assume the named tables and columns exist in your model. Replace names to match your data. Create measures using Home or Modeling → New measure. Time-intelligence examples require a proper date table and relationships.

## Lesson 17: CALCULATE()

Changes the filter context in which a measure is evaluated.

```
Uganda Commission = CALCULATE([Total Commission], REVENUE_SHARE_COMM[Country] = "Uganda")
```

**Practical note:** Use it to calculate a measure for a specific country, month, employee, or condition.

## Lesson 18: FILTER()

Returns a table containing rows that satisfy a condition.

```
High Commission = CALCULATE([Total Commission], FILTER(REVENUE_SHARE_COMM, REVENUE_SHARE_COMM[COMM] > 50000))
```

**Practical note:** Useful for row-by-row or complex filtering.

## Lesson 19: RELATED()

Retrieves a value from a related table, usually a dimension table.

```
Employee Region = RELATED(DimEmployee[Region])
```

**Practical note:** Requires a suitable relationship.

## Lesson 20: VAR & RETURN

Stores intermediate expressions to improve readability and reuse.

```
Achievement % = VAR ActualValue = [Actual Sales] VAR TargetValue = [Sales Target] RETURN DIVIDE(ActualValue, TargetValue)
```

**Practical note:** Variables make longer measures easier to maintain.

## Lesson 21: SUM()

Adds values in a numeric column.

```
Total Commission = SUM(REVENUE_SHARE_COMM[COMM])
```

**Practical note:** The result responds to report filters.

## Lesson 22: AVERAGE()

Returns the arithmetic mean of a numeric column.

```
Average Commission = AVERAGE(REVENUE_SHARE_COMM[COMM])
```

**Practical note:** DAX uses AVERAGE(); SQL commonly uses AVG().

## Lesson 23: DISTINCTCOUNT()

Counts distinct values in a column.

```
Unique Retailers = DISTINCTCOUNT(REVENUE_SHARE_COMM[RetailerID])
```

**Practical note:** Useful for unique customers, retailers, or employees.

## Lesson 24: IF()

Returns one result when a condition is true and another when false.

```
Commission Status = IF([Total Commission] >= 50000, "High", "Normal")
```

**Practical note:** Useful for simple two-outcome rules.

## Lesson 25: SWITCH()

Evaluates multiple conditions and returns the matching result.

```
Achievement Band = SWITCH(TRUE(), [Achievement %] >= 1, "Achieved", [Achievement %] >= 0.8, "Partially Achieved", "Not Achieved")
```

**Practical note:** With SWITCH(TRUE()), check thresholds from highest to lowest.

## Lesson 26: ALL()

Returns all rows or values, ignoring applicable filters on the specified table or column.

```
Overall Commission = CALCULATE([Total Commission], ALL(REVENUE_SHARE_COMM))
```

**Practical note:** ALL(Table) can remove filters from every column in that table.

## Lesson 27: ALLSELECTED()

Removes visual row/column context while retaining explicit selections such as slicers where applicable.

```
% of Selected Total = DIVIDE([Total Commission], CALCULATE([Total Commission], ALLSELECTED(REVENUE_SHARE_COMM[Employee])))
```

**Practical note:** Useful for percentage-of-selected-total calculations; behavior depends on the visual/model.

## Lesson 28: REMOVEFILTERS()

Clears filters from specified tables or columns.

```
Commission All Countries = CALCULATE([Total Commission], REMOVEFILTERS(REVENUE_SHARE_COMM[Country]))
```

**Practical note:** Other independent filters, such as Month, remain.

## Lesson 29: KEEPFILTERS()

Makes a CALCULATE filter intersect with existing filters on the same columns.

```
Uganda Commission = CALCULATE([Total Commission], KEEPFILTERS(REVENUE_SHARE_COMM[Country] = "Uganda"))
```

**Practical note:** If the current selection excludes Uganda, the intersection can be blank.

## Lesson 30: SELECTEDVALUE() & HASONEVALUE()

SELECTEDVALUE returns the single selected value or an alternate result; HASONEVALUE tests for one distinct value.

```
Selected Employee = SELECTEDVALUE(REVENUE_SHARE_COMM[Employee], "Multiple or no employees")
```

**Practical note:** Useful for dynamic titles and single-selection logic.

## Lesson 31: VALUES() & COUNTROWS()

VALUES returns distinct values in the current context; COUNTROWS counts rows in a table expression.

```
Unique Retailer Count = COUNTROWS(VALUES(REVENUE_SHARE_COMM[RetailerID]))
```

**Practical note:** DISTINCTCOUNT is simpler for many distinct-count cases.

## Lesson 32: COUNTROWS() Advanced

Counts rows in a table or table expression, often after filtering.

```
Retailers Above Threshold = COUNTROWS(FILTER(VALUES(REVENUE_SHARE_COMM[RetailerID]), CALCULATE([Total Commission]) > 50000))
```

**Practical note:** Use a distinct retailer list to count retailers rather than transaction rows.

## Lesson 33: SUMX()

Evaluates an expression for each row of a table, then sums the results.

```
Total Line Value = SUMX(FactSales, FactSales[Quantity] * FactSales[UnitPrice])
```

**Practical note:** Use when you need row-level multiplication followed by aggregation.

## Lesson 34: AVERAGEX()

Evaluates an expression per row of a table and averages the results.

```
Average Line Value = AVERAGEX(FactSales, FactSales[Quantity] * FactSales[UnitPrice])
```

**Practical note:** Averages the expression, not necessarily a single existing column.

## Lesson 35: RANKX()

Ranks items according to a measure or expression.

```
Employee Rank = RANKX(ALL(DimEmployee[Employee]), [Total Commission], , DESC, DENSE)
```

**Practical note:** ALL on the ranking category allows comparison across employees.

## Lesson 36: TOPN()

Returns the top N rows of a table based on an ordering expression.

```
Top 5 Retailers = TOPN(5, VALUES(DimRetailer[RetailerID]), [Total Commission], DESC)
```

**Practical note:** TOPN returns a table expression, not a single scalar value.

## Lesson 37: LOOKUPVALUE()

Returns a value when specified search-column conditions match.

```
Employee Rate = LOOKUPVALUE(IncentiveRates[Rate], IncentiveRates[EmployeeID],
REVENUE_SHARE_COMM[EmployeeID])
```

**Practical note:** Search conditions should identify a unique result; RELATED may be preferable with a relationship.

## Lesson 38: USERELATIONSHIP()

Activates an existing inactive relationship for one calculation.

```
Commission by Activation Date = CALCULATE([Total Commission], USERELATIONSHIP(DimDate[Date],
FactCommission[ActivationDate]))
```

**Practical note:** The relationship must already exist and be inactive.

## Lesson 39: TREATAS()

Applies values from one table expression as filters to columns in another table.

```
Selected Retailer Commission = CALCULATE([Total Commission], TREATAS(VALUES(SelectorRetailer[RetailerID]),
REVENUE_SHARE_COMM[RetailerID]))
```

**Practical note:** Useful for disconnected slicers; source and target columns must be compatible.

## Lesson 40: CROSSFILTER()

Changes relationship filter direction for the duration of a calculation.

```
Commission = CALCULATE([Total Commission], CROSSFILTER(DimEmployee[EmployeeID], FactCommission[EmployeeID],
BOTH))
```

**Practical note:** Requires an existing relationship; use bidirectional filtering carefully.

## Lesson 41: Date Table

A dedicated table with one row per date for reliable time analysis.

```
DimDate = CALENDAR(DATE(2026, 1, 1), DATE(2026, 12, 31))
```

**Practical note:** Mark it as a date table and relate its Date column to the fact date.

## Lesson 42: CALENDAR() vs CALENDARAUTO()

CALENDAR uses explicit start/end dates; CALENDARAUTO derives a range from dates in the model.

```
DimDate = CALENDAR(MIN(FactSales[OrderDate]), MAX(FactSales[OrderDate]))
```

**Practical note:** Use explicit bounds for control; CALENDARAUTO can include unexpected dates.

## Lesson 43: YEAR(), MONTH(), DAY(), QUARTER(), WEEKNUM()

Extract calendar components from a date.

```
Month Number = MONTH(DimDate[Date]); Quarter Number = QUARTER(DimDate[Date]); Year Number =
YEAR(DimDate[Date])
```

**Practical note:** Sort month names by Month Number for correct chart order.

## Lesson 44: TOTALYTD()

Calculates a year-to-date value using current date context.

```
Commission YTD = TOTALYTD([Total Commission], DimDate[Date])
```

**Practical note:** Requires a proper date table and measure.

## Lesson 45: TOTALMTD()

Calculates month-to-date values in the current date context.

```
Commission MTD = TOTALMTD([Total Commission], DimDate[Date])
```

**Practical note:** Useful for monitoring current-month progress.

## Lesson 46: TOTALQTD()

Calculates quarter-to-date values in the current date context.

```
Commission QTD = TOTALQTD([Total Commission], DimDate[Date])
```

**Practical note:** Useful for quarter progress reporting.

## Lesson 47: DATEADD()

Shifts the current date context by a specified interval.

```
Previous Month Commission = CALCULATE([Total Commission], DATEADD(DimDate[Date], -1, MONTH))
```

**Practical note:** Use a date table with a valid relationship.

## Lesson 48: SAMEPERIODLASTYEAR()

Returns the corresponding period one year earlier.

```
Commission Last Year = CALCULATE([Total Commission], SAMEPERIODLASTYEAR(DimDate[Date]))
```

**Practical note:** Useful for year-over-year comparisons when historical data exists.

## Lesson 49: PREVIOUSMONTH()

Returns the previous month's dates relative to the current date context.

```
Previous Month Commission = CALCULATE([Total Commission], PREVIOUSMONTH(DimDate[Date]))
```

**Practical note:** Requires a suitable date table and prior-period data.

## Lesson 50: Running Totals

Accumulates a measure up to the current date.

```
Cumulative Commission = CALCULATE([Total Commission], FILTER(ALL(DimDate[Date]), DimDate[Date] <=
MAX(DimDate[Date])))
```

**Practical note:** For year-to-date totals, also constrain dates to the current year.